TREND FOLLOWING OS

Control Dashboard — Architectural Document

	React 18 + Recharts	**1,857 lines · Single file**	**8 tabs · 24 pipeline scripts**

Prepared for: Systematic Trend Following Portfolio Manager
Document version: 1.0 · Date: March 2026 · Status: FINAL

1. Executive Overview

The Trend Following OS Control Dashboard is a single-file React application that serves as the operational interface for a 24-script systematic trend-following pipeline. It combines live portfolio monitoring, signal generation, multi-scenario trade recommendations, technical chart analysis, performance analytics, and backtesting validation into one cohesive, fully dark-themed terminal-style UI.

KEY DESIGN PHILOSOPHY

One file. No build pipeline. No router. No global state library. The entire application — all 8 tabs, 1,857 lines, all data, all charts — ships as a single self-contained .jsx file. It is designed to be dropped directly into the Claude.ai artifact renderer or any React sandbox.

WHAT THE DASHBOARD DOES

- **OVERVIEW tab** — displays the full strategy rule-set across 8 rule-table cards with live KPI strip
- **PIPELINE tab** — simulates pipeline execution with a live terminal log and shows the status of all 24 scripts
- **SIGNALS tab** — shows the 15 trend-qualified instruments ranked by momentum score with sector filter
- **RECOMMENDATIONS tab** — three-scenario (S1/S2/S3) rebalancing report with full execution plan per scenario
- **TECH CHARTS tab** — 4-panel Script 15 layout (price/volume/ADX/ATR) with 5 sidebar filters
- **PORTFOLIO tab** — 12 open positions with P&L, sector allocation bars, and constraint checker
- **ANALYTICS tab** — Script 24 dashboard — equity curve, monthly heatmap, drawdown, rolling Sharpe, risk metrics
- **BACKTEST tab** — Script 19 validation framework — 10 tests, 30-point scoring, red flags, WFO, Monte Carlo, AI analyst

2. Technical Stack

2.1 Core Libraries

React	18	UI framework — hooks only (useState, useEffect, useMemo, useRef)
Recharts	latest (CDN)	All data visualisations — AreaChart, BarChart, LineChart, ComposedChart, PieChart
Google Fonts (CSS)	runtime	Syne (headings), JetBrains Mono (code/data), DM Sans (body)
Anthropic API	claude-sonnet-4-20250514	AI Analyst feature in the Backtest tab (POST /v1/messages)

2.2 No-Build Architecture

The file is a JSX artifact — it runs directly in the Claude.ai artifact renderer which provides React 18 and Recharts automatically. There is no webpack, no Vite, no package.json, no node_modules, no tsconfig, and no CSS modules. All styling is done through inline JavaScript style objects.

DEPLOYMENT NOTE
To run outside Claude.ai, wrap the file in a standard Create React App or Vite project: install react, react-dom, and recharts, then import TrendFollowingOS.jsx as the root component. The Google Fonts @import in the <Fonts/> component will load from CDN automatically.

2.3 External API — Anthropic

A single API call is made from the Backtest tab's AI Analyst feature. The request is a direct fetch() POST to https://api.anthropic.com/v1/messages. In the Claude.ai environment, the API key is injected automatically. In standalone deployments, the key must be provided via an environment variable or a proxy server.

```
const response = await fetch('https://api.anthropic.com/v1/messages', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    model: 'claude-sonnet-4-20250514',
    max_tokens: 900,
    messages: [{ role: 'user', content: ANALYSIS_PROMPT }]
  })
});
```

3. File Structure

Although the application is a single file, it is logically organised into five layers that flow top-to-bottom:

1 — Imports	1–6	React hooks and Recharts chart components
2 — Design System	8–219	Fonts component, 4 palette objects (A, D, CH, P), formatting utilities, shared UI atoms
3 — Static Data	44–162	SIGNALS, PORTFOLIO, REC_SYMBOLS, EQUITY_CURVE, ROLLING_SHARPE, HEATMAP, ALLOC_PIE, PIPELINE arrays
4 — Tab Components	221–1781	OverviewTab, PipelineTab, SignalsTab, RecTab, ChartsTab, PortfolioTab, AnalyticsTab, BacktestTab
5 — App Shell	1782–end	App() root component — tab state, clock ticker, header, tab bar, content router, footer

3.1 Naming Conventions

- **UPPERCASE constants** — static data arrays (SIGNALS, PORTFOLIO, PIPELINE, EQUITY_CURVE...)
- **PascalCase components** — React components (OverviewTab, RecTab, ChartsTab, Tag, KPI, SectionBar...)
- **camelCase helpers** — pure utility functions (fN, fP, scC, genSeries, sectorColor...)
- **Single uppercase letters** — palette objects: A (app dark), D (analytics dark), CH (chart theme), P (PDF/light legacy)

4. Design System

4.1 Palette Objects

Four palette constant objects are defined at module scope. All inline styles reference these objects rather than hard-coded hex values, making global theme changes a single-point edit.

A	App Dark Palette	All 8 tabs — surfaces, borders, text	bg #04060e · card #080d16 · amber #f0b429 · text #dce6f5 · green #22d3a0 · red #f05454

D	Analytics Palette	Analytics tab only — mirrors Script 24 colours	equity #4FC3F7 · profit #69F0AE · loss #FF5252 · drawdown #FF8A65 · sharpe #CE93D8
CH	Chart Palette	Tech Charts tab — mirrors Script 15 exact hex	price #2563EB · sma50 #F59E0B · sma200 #EF4444 · adx #8B5CF6 · atr #F97316 · stop #DC2626
P	PDF Legacy Palette	Retired — was used when RecTab rendered a white paper PDF mockup. No longer referenced.	navy #1B2A47 · accent #2E86DE · success #27AE60 · danger #E74C3C

4.2 Typography

Syne	Display / Headings	Tab names, section headers, KPI labels, action tile counts, app title, all UPPERCASE labels
JetBrains Mono	Monospaced / Data	All numeric values, ticker symbols, dates, pipeline logs, code snippets, filter buttons
DM Sans	Body / UI	Descriptive text, subtitles, chart tooltips, sector names, filter panel labels, navigation

4.3 Shared UI Atoms

Six reusable micro-components are defined at module scope and used across all tabs:

<Fonts/>	none	Injects Google Fonts @import + global CSS reset + animations (fadeIn, blink) + scrollbar overrides into a <style> tag
<Tag/>	children, c, sm	Small inline badge with coloured border and translucent background. c sets colour, sm makes it compact.
<KPI/>	l, v, s, c	Metric card with label (l), value (v), subtitle (s), and accent colour (c). Used in strip rows at the top of most tabs.
<SectionBar/>	c	3px × 16px accent bar used as a visual prefix on section headings. c sets the bar colour.
<ChartTip/>	active, payload, label, pct	Custom Recharts tooltip component. Renders a dark panel. pct=true formats values as percentages.

genSeries()	sym, base, drift, n	Deterministic OHLCV series generator. Uses symbol name as seed for reproducibility. Returns n data points with close, sma50, sma200, vol, volAvg, adx, atr fields.

4.4 Animations

- **.anim class** — fadeIn keyframe — every tab wrapper uses className='anim' for a 300ms slide-up entrance on mount
- **.blink class** — step-start blink keyframe — used on cursor characters in the Pipeline terminal log and loading states

> 💡 *Animations are defined entirely in the <Fonts/> component's injected <style> block — no external CSS file is needed.*

5. Static Data Layer

All data is declared as module-scope constants. In a production system, these would be fetched from API endpoints or server-side files; in this single-file architecture they are hardcoded representative values that mirror the real pipeline outputs.

SIGNALS	15	Trend-qualified instruments from Script 07. Fields: rank, symbol, name, exch, assetClass, sector, close, sma50, sma200, adx, atrP, score, r20, r60, r120
PORTFOLIO	12	Open positions from Script 11. Fields: symbol, assetClass, sector, shares, entry, curr, stop, stopType, days, entryDate, name
REC_SYMBOLS	3 keys	Symbol lists per recommendation scenario: {s1: [...], s2: [...], s3: [...]}. Drives the recommendation filter in ChartsTab.
ALL_REC_SYMBOLS	derived	Flat deduplicated union of all REC_SYMBOLS values (Set). Used for the 'Any rec' filter option.
EQUITY_CURVE	63 months	IIFE-generated — iterates monthly strategy/benchmark returns to build cumulative NAV and drawdown series for analytics charts.
ROLLING_SHARPE	51 months	Derived from EQUITY_CURVE — 12-month rolling Sharpe calculated with annualisation factor √12 and a 0.28 risk-free offset.
HEATMAP	6 years	Object keyed by year (2020–2025), each value an array of 12 monthly return % values (null = future month).

ALLOC_PIE	4 slices	Asset class allocation for the Analytics donut chart: [{name, value, color}, ...]
PIPELINE	23 entries	All 24 pipeline scripts (script 13 omitted) with num, label, mode, lastRun, status, dur. Powers both the pipeline table and the mode simulation logic.

DATA FLOW NOTE

There is no prop-drilling between tabs. Each tab imports the module-scope data constants directly — they behave like a shared in-memory store. This is intentional for a single-file architecture. In a multi-file project, these would be moved to a shared data/ or store/ module.

6. Tab Components

All 8 tab components follow the same structural pattern: a top-level wrapper, an optional KPI strip row, then section content. Each component is fully self-contained — its internal helpers (table formatters, sub-components) are declared as closures within the function body.

6.1 OverviewTab

STATE & HOOKS

None. This tab is entirely stateless — it renders a fixed strategy blueprint.

LAYOUT

- KPI strip — 8 metric cards in a flex-wrap row (CAGR, Sharpe, Max DD, Deploy status, etc.)
- Rule grid — 8 cards in a responsive CSS grid (minmax 375px). Each card maps to one pipeline phase.

RULE CARDS COVERED

- **01–02 · Data Layer** — Sources, universe, frequency, minimum history
- **04 · Universe Screen** — Price, liquidity, market cap, ETF filters
- **05–06 · Trend Qualify** — SMA50 > SMA200, Close > SMA50, ADX14 > 20 — all three required
- **07 · Momentum Rank** — Score = ((Close − SMA200) / SMA200) × 100, aux ROC fields
- **08 · Stop-Loss** — Initial = Entry − 3×ATR, trailing after +15%, Fridays only, ratchet-up
- **09 · Position Sizing** — 2% target risk, inverse-ATR vol scaling, 0.5%–8% equity clip
- **11 · Circuit Breakers** — DD < −15%, VIX ≥ 40, correlation > 0.85, concentration, staleness
- **11 · Rebalancing** — Monthly / weekly / daily cadence, execution order

6.2 PipelineTab

STATE

- **run** — string | null — ID of currently executing pipeline mode, null when idle
- **log** — array of script entries appended via setInterval during simulation
- **lRef** — useRef pointing to the log scroll container — auto-scrolled via useEffect

EXECUTION SIMULATION

Clicking a mode button starts an interval (500 ms per step) that filters the PIPELINE array by mode and appends one entry per tick. The interval self-clears when all steps are logged. This simulates the real Python subprocess output without requiring a backend.

MODE MAPPING

DAILY	1, 3, 14	~1 min	Download + monitor
WEEKLY	1, 3, 9, 10, 14	~2–8 hr	Stops + exit signals
MONTHLY	1–12, 15, 22–24	~2–12 hr	Full rebalance cycle
BACKTEST	16–21	hours	Full validation pipeline
ANALYTICS	22, 23, 24	~30 sec	Performance reporting
VALIDATION	19, 20, 21	~5 min	Re-run validator only
REPORT	12	~2 min	Regenerate PDF report

6.3 SignalsTab

STATE

- **sec** — string — active sector filter ('ALL' or sector name)

BEHAVIOUR

Renders all 15 entries from SIGNALS filtered by sector. Columns match the Script 07 output exactly: rank, symbol, exchange, sector, close, SMA50, SMA200, ADX, ATR%, momentum score, ROC20/60/120. ADX values > 30 render in green (strong trend), amber otherwise. Score values > 25 render in green.

6.4 RecTab (Recommendations)

STATE

- **activeScen** — 0 = Comparison view, 1/2/3 = individual scenario

INTERNAL CONSTANTS

- **BG / CARD / CARD2 / BDR / BDR2** — local dark palette overrides that are strictly darker than A — near-black surfaces for the paper report feel

INTERNAL HELPERS

- **SecHead** — renders a coloured SectionBar + Syne uppercase label — the dark equivalent of the old PDF section headers
- **th() / tdR()** — table cell style factories for headers and data rows; tdR() takes an alt boolean for alternating row shading
- **kvRow()** — two-cell key-value row for the capital summary and meta tables
- **DCard** — dark card component with optional accent left-border, optional titled header band, and

consistent padding

- **ActionTile** — large count tile (Exits / Entries / Holds) with translucent tinted background using the scenario's accent colour
- **SharedExits** — inline sub-component — Mandatory Exits + Rotation Exits tables; identical markup in both Comparison and individual scenario views
- **HoldsTable** — inline sub-component — 10 hold positions table
- **CBTable** — inline sub-component — 5-row circuit breaker status table

THREE SCENARIOS

The SCENARIOS array holds three objects. Each differs only in: color (accent), entries (3 new positions), assetBreakdown, cashAfter, and warnings. The comparison view renders all three in a grid; the individual view renders the selected scenario's full execution plan with checklist and warnings.

6.5 ChartsTab (Technical Charts)

STATE

- **sel** — string — currently selected symbol (defaults to SIGNALS[0].symbol)
- **lb** — number — lookback window in days (60 / 120 / 200)
- **search** — string — live symbol/company name search query
- **fAsset** — string — asset class filter ('ALL' | 'US Stock' | 'EU Stock' | 'Cryptocurrency' | 'ETF')
- **fExch** — string — exchange filter ('ALL' | 'NYSE' | 'NASDAQ' | 'XETRA' | 'CRYPTO' | 'ETF')
- **fPort** — boolean — when true, shows only symbols that appear in PORTFOLIO
- **fRec** — string — recommendation filter ('ALL' | 'S1' | 'S2' | 'S3' | 'ANY')

USEMEMO OPTIMISATION

- **filtered** — the filtered SIGNALS list — recomputed only when any filter state changes
- **series** — the OHLCV series for the selected symbol — genSeries() is expensive so memoised on [sel, lb]

AUTO-SELECTION ON FILTER

A useEffect monitors filtered. If the currently selected symbol disappears from the filtered list (due to a filter change), it auto-selects the first visible symbol. This prevents the chart from going blank.

LAYOUT

Fixed two-panel layout: a 232px dark sidebar on the left (filter panel + symbol list) and a full-height chart area on the right. Height is calc(100vh - 160px) so the chart fills the viewport.

FOUR CHART PANELS

- **Price area (60% height)** — ComposedChart — price Area + SMA50 Line + SMA200 dashed Line + stop ReferenceLine + entry ReferenceLine
- **Volume bar (20% height)** — ComposedChart — volume Bar + 50-day average Line
- **ADX area (10% height)** — AreaChart — ADX(14) area with 20-threshold ReferenceLine
- **ATR% area (10% height)** — AreaChart — ATR%(20) area, values formatted as percentages

6.6 PortfolioTab

STATE & HOOKS

No state. Computed values (total portfolio value, sector allocation map) are derived inline via .reduce() on the PORTFOLIO constant.

LAYOUT

- KPI strip — 5 metric cards (open positions, total value, unrealised P&L, trailing stop count, crypto weight)
- Full-width positions table with 14 columns (including stop type tag)
- Two-column lower section: sector allocation bar chart (custom CSS bars) + constraint checker

CONSTRAINT CHECKER

7 live constraints checked and colour-coded: cash ≥ 5%, crypto ≤ 20%, sector ≤ 30%, max position ≤ 8%, top-3 ≤ 30%, pairwise correlation, data staleness. Values that breach thresholds render in red with a WARN tag.

6.7 AnalyticsTab

STATE & HOOKS

No state. All data comes from EQUITY_CURVE, ROLLING_SHARPE, PORTFOLIO, and HEATMAP module-scope constants. The D palette (Analytics dark theme) is used exclusively in this tab to match Script 24's colour scheme.

PANELS

- **Metric strip** — 8 KPI cards with Script 24 colours (profit green, loss red)
- **Risk analytics strip** — 8 inline metrics from Script 23 (VaR 95/99, CVaR, Beta, TE, IR, HHI, Effective N)
- **Equity curve** — AreaChart — strategy NAV area + SPY benchmark dashed line, €K-formatted Y-axis
- **Monthly heatmap** — HTML <table> with 6 rows × 12 columns — cells shaded from deep red (< −4%) through greens
- **Drawdown chart** — AreaChart with max DD reference line at −19.8%
- **Rolling Sharpe chart** — LineChart with 1.0 threshold reference line
- **Asset allocation donut** — PieChart with 4 slices, formatted €K tooltip
- **Position P&L table** — Full position list with per-row P&L colouring (profit = green, loss = red)

6.8 BacktestTab

STATE

- **aiR** — string — AI analyst response text (empty = not yet run)
- **aiL** — boolean — loading state for the API call

VALIDATION DATA

- **TESTS** — 10 primary validation tests (T01–T10) with pass/fail, actual value, threshold
- **SCORE** — 10 scoring metrics, each 0–3 points, summing to 27/30 (rating: GOOD)
- **RF** — 7 critical red flags (RF01–RF07), all clear
- **BM** — 3 benchmark comparisons (SPY, 60/40, Naive Trend) with CAGR/Sharpe/MaxDD
- **MCd** — 25-point Monte Carlo fan data (P5/P25/P50/P75/P95 paths over 60 months)

AI ANALYST — API CALL

The 'RUN ANALYSIS' button calls the Anthropic API with a hardcoded prompt containing the full backtest statistics, strategy rules, and OOS results. The response is displayed as pre-wrapped text below the button. Error state is handled with a try/catch. The model is pinned to claude-sonnet-4-20250514.

7. App Shell — App() Component

The default export App() is the root component. It owns all navigation state and renders the persistent chrome (header, tab bar, footer) around the active tab content.

7.1 State

- **tab** — string — active tab ID, defaults to 'overview'
- **time** — Date — current time, updated by a 1-second setInterval (cleared on unmount)

7.2 Tab Registry

Two parallel arrays define the tab system: TABS (an array of {id, icon, label} objects) and C (an object mapping tab ID to its JSX element). Switching tabs is a simple setState — there is no router, no lazy loading, and no unmount/remount — all 8 components remain mounted in the component tree.

PERFORMANCE NOTE

All 8 tab components are always mounted. This is intentional — it avoids the state-reset that would occur if tabs were conditionally rendered. Charts and data are always available. For a large dataset, consider React.memo() wrappers or lazy(() => import()) if needed.

7.3 Header

- Left: animated logo strip (3 amber bars of decreasing height) + app title + architecture version
- Right: live clock with market status, and three status tags (● LIVE, 12 POS, ✓ GO)

7.4 Tab Bar

A horizontal flex row of 8 <button> elements. Active tab gets an amber bottom border and the A.amber colour. Each button renders an icon character + Syne uppercase label. The bar overflows horizontally on narrow viewports.

7.5 Footer

A dark footer strip with the disclaimer, universe stats (892 screened, 15 qualified), portfolio count, last rebalance date, and next execution date.

8. Chart Architecture (Recharts)

All charts use Recharts with ResponsiveContainer to fill their parent width. The chart backgrounds match the surrounding dark surface — there is no chart-internal background colour. Grids use A.border or D.grid. Tooltips use the shared ChartTip component.

AreaChart + Area	Charts (price, ADX, ATR), Analytics (equity, DD)	Linear gradient fills with low opacity (0.14–0.38). stroke = solid. fill = url(#gradientId)
ComposedChart	Charts (price panel, volume panel)	Mixes Area + Line + Bar types. Essential for overlaying SMA lines on the price area.
LineChart + Line	Analytics (rolling Sharpe), Backtest (WFO)	strokeDasharray on benchmark/ reference lines. dot={false} for performance.
BarChart + Bar	Charts (volume), Analytics, Backtest (WFO)	radius={[2,2,0,0]} on all bars for rounded top corners. Cell component used for per-bar colouring.
PieChart + Pie	Analytics (allocation donut)	innerRadius + outerRadius creates donut shape. paddingAngle=2. stroke=D.panelBg creates gap effect.
ReferenceLine	Charts (stop, entry), Analytics (max DD, Sharpe=1)	strokeDasharray for dashed lines. label prop with position/fill/fontSize for inline annotation.

8.1 genSeries() — Chart Data Generator

Since live market data is not fetched, the chartsTab uses a deterministic pseudo-random series generator. The function takes a symbol name, base price, drift, and point count as inputs. The symbol name is used as a numeric seed to ensure consistent chart appearance across re-renders.

```
const genSeries = (sym, base, drift, n=190) => {
  // seed from symbol chars: sym.charCodeAt(0)*0.6 + sym.charCodeAt(1)*0.4
  // each bar: p *= (1 + drift*0.0018 + noise + sin_wave)
  // computes rolling sma50, sma200, volAvg as the series builds
  // returns array of: { date, close, hi, lo, vol, sma50, sma200, volAvg, adx, atr }
}
```

💡 *The drift parameter is set to 1 + (rank_index × 0.04) per symbol so higher-ranked instruments show stronger uptrends, visually consistent with the momentum rank.*

9. Recommendations Tab — Deep Dive

The Recommendations tab is the most complex component in the application. It replicates the multi-scenario structure of the real Script 12 PDF output in a fully interactive dark UI.

9.1 Scenario Data Structure

```
const SCENARIOS = [
  {
    id: 1,
    name: 'Scenario 1 — Pure Momentum',
    shortName: 'S1  PURE MOMENTUM',
    philosophy: '...',
    candidatePool: '...',
    color: '#4ea8f0',          // accent colour used for headers, borders, tags
    entries: [                  // array of 3 new position objects
      { rk, sy, nm, ex, sc, sh, lp, pv, pp, st, sd, adx, atr, ms }
    ],
    newCount, rotCount, mandCount, holdCount,
    exitProceeds, entryRequired, cashAfter, cashEur,
    assetBreakdown: [['CLASS', '€val', '%', 'n'], ...],
    warnings: ['...'],
  },
  // ...S2 and S3 follow the same shape
]
```

9.2 Shared vs. Scenario-Specific Content

Circuit breakers	Shown once (CBTable)	Shown once (CBTable)
Mandatory exits (2)	Shown once (SharedExits)	Shown once (SharedExits)
Rotation exits (1)	Shown once (SharedExits)	Shown once (SharedExits)
Holds (10)	Shown once (HoldsTable)	Shown once (HoldsTable)
New entries	Summary only (3 rows per card)	Full 14-column table (scenario-specific)
Capital summary	Footer of each card	Full table (scenario-specific)
Asset breakdown	Not shown	Full table (scenario-specific)
Execution checklist	Not shown	10-item grid (scenario-specific)
Warnings	Cash/sector note in card	Full warning banner (scenario-specific)

10. Tech Charts Filter System

The ChartsTab sidebar contains a five-layer filter system that narrows the symbol list from 15 to any subset. All filters compose with AND logic — a symbol must pass every active filter to be shown.

Search	search (string)	Any text input	Case-insensitive match on symbol OR company name
Asset Class	fAsset (string)	ALL / US Stock / EU Stock / Cryptocurrency / ETF	Exact match on SIGNALS[].assetClass
Exchange	fExch (string)	ALL / NYSE / NASDAQ / XETRA / CRYPTO / ETF	Exact match on SIGNALS[].exch
Portfolio	fPort (boolean)	All / Held positions only	Passes if PORTFOLIO includes the symbol
Recommendations	fRec (string)	ALL / S1 / S2 / S3 / ANY	S1/S2/S3: symbol in that scenario's entries. ANY: symbol in any scenario.

The filtered list is computed with useMemo() and drives both the sidebar symbol list and the auto-selection useEffect. A 'Clear all' button resets all five filters to their defaults simultaneously.

> 💡 *Recommendation filter badges (S1, S2, S3) are rendered on each symbol row in the sidebar using the scenario accent colours — blue, green, purple respectively. The HELD badge is always rendered in green when the symbol appears in PORTFOLIO.*

11. How to Build and Extend

11.1 Adding a New Tab

To add a ninth tab, follow these three steps:

```
// 1. Define the component
const MyNewTab = () => {
  return <div className='anim'>...</div>;
};

// 2. Register it in the TABS array inside App()
{ id: 'mynewtab', icon: '◉', label: 'MY TAB' }

// 3. Add it to the C content router inside App()
mynewtab: <MyNewTab />,
```

11.2 Connecting Live Data

To replace static data with real pipeline output, swap the module-scope constants with useEffect-driven fetches. The recommended pattern is to keep the same constant names as fallback values, then overwrite them with server data on mount:

```
// Before (static):
const SIGNALS = [{ rank:1, symbol:'NVDA.US', ... }, ...];

// After (live):
const [signals, setSignals] = useState(SIGNALS_FALLBACK);
useEffect(() => {
  fetch('/api/signals').then(r => r.json()).then(setSignals);
}, []);
```

11.3 Adding Real Chart Data

Replace genSeries() calls with OHLCV API fetches. The Recharts components expect the same field names (date, close, sma50, sma200, vol, volAvg, adx, atr), so the chart JSX requires no changes — only the data source changes.

11.4 Splitting into Multiple Files

The natural split points are: (1) data/ directory for all SIGNALS, PORTFOLIO, etc. constants; (2) components/shared/ for Tag, KPI, SectionBar, ChartTip; (3) components/tabs/ for each tab file; (4) styles/palette.js for A, D, CH objects. The App.jsx root imports all tabs and renders the shell.

11.5 Extending the Recommendation Scenarios

The SCENARIOS array in RecTab is data-driven — adding a fourth scenario requires only adding a new object to the array with the same shape. The scenario selector tab bar, comparison grid, and individual view all render dynamically from the array length.

11.6 Securing the Anthropic API Key

In production, never expose an API key in client-side code. Route the Backtest AI Analyst call through a backend proxy endpoint that injects the key server-side. Replace the direct fetch() to api.anthropic.com with a call to your own /api/analyze endpoint.

12. Key Implementation Patterns

12.1 Inline Style Pattern

All styles are plain JavaScript objects. There are no CSS classes (except the animation classes .anim and .blink defined in <Fonts/>). The advantages are: no style collision, no specificity issues, full dynamic colouring from palette variables, and zero external CSS dependencies.

```
// Pattern: spread + override
style={{ ...td('right', true), color: A.green, fontWeight: 700 }}

// Pattern: conditional colour
color: sig.adx > 30 ? A.green : A.amber

// Pattern: translucent tint from hex colour
background: `${scen.color}18`   // 18 = hex opacity ~10%
border: `1px solid ${scen.color}40`  // 40 = hex opacity ~25%
```

12.2 Style Factory Functions

Repetitive style objects are extracted into factory functions scoped inside the component that uses them. This avoids object recreation on every render while keeping the style logic close to its usage.

```
// Inside RecTab — style factories
const th  = (c, al='right') => ({
  background: CARD2, color: c, fontSize: 9, fontWeight: 700,
  padding: '6px 8px', textAlign: al, borderBottom: `2px solid ${c}`,
  whiteSpace: 'nowrap', fontFamily: 'JetBrains Mono',
});
const tdR = (al='right', alt=false) => ({
  fontSize: 9, padding: '5px 8px', borderBottom: `1px solid ${BDR}`,
  textAlign: al, background: alt ? CARD2 : CARD, color: '#c8d8f0',
  fontFamily: 'JetBrains Mono',
});
```

12.3 Inline Sub-Components

Complex repeated sections within a tab are defined as inner functional components (using const declarations inside the tab function). This is valid React — they re-render with the parent. It keeps related markup collocated and avoids prop-drilling while still making the JSX readable.

```
// Inside RecTab — inline sub-component
const SharedExits = () => (
  <>
    <SecHead c={A.red}>Mandatory Exits (2)</SecHead>
    <table>...</table>
    <SecHead c={A.orange}>Rotation Exits (1)</SecHead>
    <table>...</table>
  </>
);
// Used in both Comparison and Individual views:
<SharedExits/>
```

12.4 IIFE for Derived Data

The EQUITY_CURVE constant uses an Immediately Invoked Function Expression (IIFE) to compute a derived dataset at module load time. This keeps the data declaration at module scope (available to all tabs) while allowing complex multi-step derivation logic:

```
const EQUITY_CURVE = (() => {
  const r = []; let s = 100000, b = 100000, pk = 100000;
  [[label, stratRet, benchRet], ...].forEach(([l, sr, br]) => {
    s *= (1+sr); b *= (1+br); pk = Math.max(pk, s);
    r.push({ label:l, strategy:Math.round(s), benchmark:Math.round(b),
             dd: parseFloat(((s/pk-1)*100).toFixed(2)) });
  });
  return r;
})();
```

13. Quick-Start Guide

Option A — Claude.ai Artifact (Recommended)

- Open a new conversation with Claude

- Paste the entire TrendFollowingOS.jsx file content into a message asking Claude to render it as an artifact
- Or: upload the .jsx file and ask 'render this as a React artifact'
- The artifact renderer provides React 18, Recharts, and Google Fonts automatically
- The Anthropic API key is injected automatically for the AI Analyst feature

Option B — Vite Project

```
npm create vite@latest trend-os -- --template react
cd trend-os
npm install recharts
# Copy TrendFollowingOS.jsx to src/
# Replace src/App.jsx contents with:
import TrendFollowingOS from './TrendFollowingOS';
export default TrendFollowingOS;
# Add API key to .env:
VITE_ANTHROPIC_KEY=sk-ant-...
# Update the fetch() headers in BacktestTab to include the key
npm run dev
```

Option C — Create React App

```
npx create-react-app trend-os
cd trend-os
npm install recharts
# Copy TrendFollowingOS.jsx to src/
# In src/index.js replace App import with TrendFollowingOS
npm start
```

NOTE ON API KEY IN STANDALONE BUILDS

The AI Analyst button in the Backtest tab calls the Anthropic API directly from the browser. In production builds outside Claude.ai, you MUST proxy this through a backend server to avoid exposing the secret key. Never commit an API key to source control.

14. Full Component Tree

The complete component hierarchy of the application:

```
App()
├── <Fonts/>                        global CSS injection
├── Header                          sticky top bar
│   ├── Logo strip (3 amber bars)
│   ├── App title + version
│   └── Status tags (LIVE / POS / GO) + clock
├── Tab bar                         8 navigation buttons
├── Content area  [active tab only visible]
│   ├── <OverviewTab/>
│   │   ├── KPI strip  (8× <KPI/>)
```

```
│   │   └── Rule grid  (8 rule cards)
│   ├── <PipelineTab/>
│   │   ├── Mode buttons  (7× button)
│   │   ├── Terminal log
│   │   └── Scripts table (23 rows)
│   ├── <SignalsTab/>
│   │   ├── KPI strip
│   │   ├── Sector filter bar
│   │   └── Signals table (15 rows)
│   ├── <RecTab/>
│   │   ├── Scenario tab bar (Comparison + 3 scenarios)
│   │   ├── Comparison view  [activeScen === 0]
│   │   │   ├── Meta strip
│   │   │   ├── 3× scenario cards
│   │   │   ├── Side-by-side metrics table
│   │   │   ├── <CBTable/>
│   │   │   ├── <SharedExits/>
│   │   │   └── <HoldsTable/>
│   │   └── Individual scenario view  [activeScen > 0]
│   │       ├── Scenario hero header
│   │       ├── 4× <ActionTile/>
│   │       ├── <CBTable/>
│   │       ├── <SharedExits/>
│   │       ├── New Entries table
│   │       ├── <HoldsTable/>
│   │       ├── Capital summary (2 col)
│   │       ├── Execution checklist
│   │       └── Warning banners
│   ├── <ChartsTab/>
│   │   ├── Sidebar (232px)
│   │   │   ├── Lookback buttons (60/120/200)
│   │   │   ├── Search input
│   │   │   ├── Asset class filter
│   │   │   ├── Exchange filter
│   │   │   ├── Portfolio filter
│   │   │   ├── Rec scenario filter
│   │   │   └── Symbol list (filtered)
│   │   └── Chart area
│   │       ├── Chart header (metrics + qualification badges)
│   │       ├── Price ComposedChart
│   │       ├── Volume ComposedChart
│   │       ├── ADX AreaChart
│   │       └── ATR% AreaChart
│   ├── <PortfolioTab/>
│   │   ├── KPI strip
│   │   ├── Positions table (12 rows)
│   │   ├── Sector allocation bars
│   │   └── Constraint checker
│   ├── <AnalyticsTab/>
│   │   ├── Metric strip (8 cards)
│   │   ├── Risk analytics strip
│   │   ├── Equity curve AreaChart
│   │   ├── Monthly heatmap table
│   │   ├── Drawdown AreaChart
│   │   ├── Rolling Sharpe LineChart
│   │   ├── Asset allocation PieChart
│   │   └── Position P&L table
│   └── <BacktestTab/>
│       ├── Rating banner
│       ├── 10-test pass/fail table
│       ├── 30-pt scoring table
│       ├── 7 red flags table
│       ├── Benchmark comparison + WFO BarChart
```

```
│       ├── Monte Carlo AreaChart
│       └── AI Analyst panel
└── Footer
```

End of Document — Trend Following OS Architectural Document v1.0 — March 2026